

Introduction

Here I document how I built this project from the beginning.

I started learning computer networking from **Computer Networking: A Top-Down Approach (9th Edition)**. The second chapter introduces socket programming using Python, but I felt that many implementation details were hidden behind Python's abstractions. I wanted to build the same concepts in C++ instead. While doing that, I discovered that I had to rely on the POSIX socket API, which is written in C. Since I wasn't familiar with it, I decided to first get the big picture of socket programming by reading **Beej's Guide to Network Programming**.

The main concepts I learned from it were the following:

1. Sockets

A socket is an operating system resource used for network communication. In Unix-like systems, it is represented by a **file descriptor**, which is a non-negative integer assigned by the operating system to an open file or other I/O resource. Programs use this file descriptor to perform input and output operations on the socket.

2. Host and Network Byte Order

Different CPU architectures may store multi-byte values in memory differently. Most modern Intel and AMD processors use **Little-Endian** byte order, while Internet protocols use **Big-Endian**, commonly referred to as **Network Byte Order**.

For example, the 16-bit hexadecimal value `0xB34F` is stored differently depending on the byte order.

In **Big-Endian**, the most significant byte comes first:

B3 4F

In **Little-Endian**, the least significant byte comes first:

4F B3

Because different systems may use different byte orders, applications should not assume that the host byte order matches the network byte order. Before sending numeric values over the network, they should be converted to Network Byte Order. Likewise, values received from the network should be converted back to the host's native byte order.

The standard conversion functions are:

Function	Description
<code>htons()</code>	h ost to n etwork s hort
<code>htonl()</code>	h ost to n etwork l ong
<code>ntohs()</code>	n etwork to h ost s hort
<code>ntohl()</code>	n etwork to h ost l ong

As Beej's Guide to Network Programming puts it:

"Basically, you'll want to convert the numbers to Network Byte Order before they go out on the wire, and convert them to Host Byte Order as they come in off the wire."

3. Converting an IP Address to a Readable Format

IP addresses are stored internally in binary form. To convert an IP address into a human-readable string such as `"192.168.1.1"`, we use `inet_ntop()` (**Network to Presentation**).

4. Resolving Hostnames

Before creating a connection, we often need to resolve a hostname (such as `google.com`) into one or more IP addresses.

To do this, we use the `<netdb.h>` header, which provides the `getaddrinfo()` function.

`getaddrinfo()` takes a hostname, a service (or port), and a set of **hints** (metadata describing the type of address we want). It then returns a linked list of matching addresses.

Each node in this linked list is an `addrinfo` structure.

`addrinfo` is a structure that stores information about a network address. It is used by `getaddrinfo()` to return one or more addresses that can be used to create a socket and either connect to a remote host or bind a server socket.

```
struct addrinfo {
    int          ai_flags;      // AI_PASSIVE, AI_CANONNAME, etc.
    int          ai_family;     // AF_INET, AF_INET6, AF_UNSPEC
    int          ai_socktype;   // SOCK_STREAM, SOCK_DGRAM
    int          ai_protocol;   // use 0 for "any"
    size_t       ai_addrlen;    // size of ai_addr in bytes
    struct sockaddr *ai_addr;    // struct sockaddr_in or _in6
    char         *ai_canonname; // full canonical hostname

    struct addrinfo *ai_next;    // linked list, next node
};
```

`addrinfo` contains several members that describe a network address and how it should be used.

- `ai_flags`: Specifies how you want the address lookup to behave
 - `AI_PASSIVE`: When the hostname is `nullptr`, `getaddrinfo()` returns a wildcard address, allowing a server to listen on all available network interfaces.

- `ai_family` : Specifies which IP version to use (IPv4, IPv6, or either).
- `ai_socktype` : Specifies the socket type, such as `SOCK_STREAM` (TCP) or `SOCK_DGRAM` (UDP).
- `ai_addr` : Points to the actual network address. Internally, it may point to either a `sockaddr_in` (IPv4) or a `sockaddr_in6` (IPv6) structure.
- `ai_canonname` : Stores the canonical hostname. It is only filled when `AI_CANONNAME` is specified in `ai_flags`.
- `ai_next` : Points to the next `addrinfo` structure. Since `getaddrinfo()` returns a linked list of addresses, this member allows you to iterate through them.

For example, if you want to create a TCP server listening socket, you can pass the following metadata to `getaddrinfo()` :

```
addrinfo metadata{};
metadata.ai_flags = AI_PASSIVE;
metadata.ai_socktype = SOCK_STREAM;
metadata.ai_family = AF_UNSPEC;
```

I wasn't a fan of working directly with the C API, so I wrapped it in a namespace called `address_resolver`.

It contains a `resolve()` function that returns a `resolved_addresses` object. This wrapper provides helper functions such as `IP()` and `Port()`, which internally use `inet_ntop()` and `ntohs()`. It also automatically calls

`freeaddrinfo()` in its destructor, so the allocated resources are released using RAII.

5. Creating a Socket

To create a socket, we include `<sys/socket.h>` and call `socket()`. It takes the address family and socket type as arguments and returns a file descriptor (`int`) on success or `-1` on failure.

`errno` is a global variable that stores error information for many system calls, such as `socket()` and `listen()`.

To get a human-readable description of the error, we can use `perror()` or `strerror()`.

6. Binding a Socket

`bind()` associates a socket with a network address.

Like many POSIX networking functions, it returns `-1` on failure and sets `errno` to indicate the cause of the error.

7. Connecting and Listening

For a client, we usually don't care which local port is used. The operating system automatically chooses an available

Ephemeral port (temporary or short-lived) when `connect()` is called, so we normally don't call `bind()` for client sockets.

The server, however, must call `listen()` to accept incoming connection requests.

`listen()` takes a **backlog** parameter, which specifies how many incoming connection requests can remain pending in the listen queue.

When a client connects, the server calls `accept()`. This creates a **new socket** dedicated to communicating with that client, while the original listening socket continues accepting new connections.

8. Sending and Receiving Data

TCP communication uses `send()` and `recv()`.

Neither function guarantees that all requested bytes are transferred in a single call. When sending or receiving large amounts of data, you should keep calling them until all the intended data has been processed.

Both functions return `-1` on error.

`recv()` can also return `0`, which indicates that the remote peer has closed the connection.

The data buffer is passed as a `void*` because these functions are designed to work with any type of data. The pointer is simply treated as a sequence of bytes.

9. UDP Communication

UDP uses `sendto()` and `recvfrom()` instead of `send()` and `recv()`.

Unlike TCP, UDP is connectionless, so each datagram must include the destination address when sending, and `recvfrom()` also provides the sender's address when receiving.

10. `close()` vs `shutdown()`

`close()` releases the socket's file descriptor.

`shutdown()` disables communication without immediately releasing the file descriptor.

It supports three modes:

- `SHUT_RD` - disables receiving (read).

- `SHUT_WR` - disables sending (write).
- `SHUT_RDWR` - disables both sending and receiving.

Although `shutdown()` is optional, every socket should eventually be closed using `close()`.

Why use `shutdown()` ?

Suppose one thread is blocked waiting for data:

Thread A

```
recv(sockfd, ...); // blocked
```

While another thread calls:

Thread B

```
close(sockfd);
```

Calling `close()` directly releases the file descriptor while another thread is still using it, which can lead to confusing or unsafe behavior.

Instead, Thread B calls `shutdown()` first. This causes the blocked `recv()` in Thread A to wake up and return `0`. After that, we call `join()` to wait for the receiver thread to exit

cleanly, and finally call `close()` to release the file descriptor.

11. Wrapping the Socket API

Rather than calling the POSIX socket functions directly throughout the project, I wrapped them inside a `socket_handler` class. This simplified resource management and provided a cleaner C++ interface.

12. Blocking vs Non-Blocking Sockets

In programming, **blocking** is common jargon for **waiting**. For example, when `recv()` is called and no data is available, the calling thread blocks (sleeps) until data arrives.

For a server, especially the listening socket, blocking behavior is usually undesirable because it prevents the server from handling other events efficiently.

To make a socket non-blocking, we use the `<fcntl.h>` header.

```
fcntl(sockfd, command, flags);
```

Some commonly used commands are:

- `F_GETFL` - retrieves the current file status flags.

- `F_SETFL` - updates the file status flags.

To enable non-blocking mode, we first retrieve the current flags, then set the `O_NONBLOCK` flag:

```
flags = fcntl(sockfd, F_GETFL); fcntl(sockfd,
F_SETFL, flags | O_NONBLOCK);
```

Once a socket is non-blocking, operations such as `recv()` no longer wait for data. If no data is available, `recv()` immediately returns `-1`, and `errno` is set to either `EAGAIN` or `EWOULDBLOCK`, indicating that no data is currently available.

13. `poll()`

`poll()` is a synchronous I/O multiplexing mechanism provided by `<poll.h>`.

The idea is to pass an array of `pollfd` structures representing the file descriptors you want to monitor.

The kernel checks each file descriptor in the array and returns the number of descriptors that are ready for the requested operations.

Conceptually, you can think of it as the kernel asking each file descriptor:

"Are you ready?"

A `pollfd` structure contains three main members:

- `fd` - the file descriptor to monitor.
- `events` - the events you're interested in.
- `revents` - the events that actually occurred.

Macro	Description
<code>POLLIN</code>	Alert me when data is ready to <code>recv()</code> on this socket.
<code>POLLOUT</code>	Alert me when I can <code>send()</code> data to this socket without blocking.
<code>POLLHUP</code>	Alert me when the remote closed the connection.

Edge Triggered vs Level Triggered

`epoll` supports two notification modes:

- **Level Triggered (LT)** (*the default*): As long as data remains available to read, `epoll` continues reporting the socket as ready.
- **Edge Triggered (ET)**: `epoll` reports the socket only when its state changes from "not ready" to "ready." If you don't read all available data, you won't receive another notification until new data arrives.

Because of this, when using Edge Triggered mode, you should keep calling `recv()` until it returns `-1` and `errno` is set to `EAGAIN` or `EWOULDBLOCK`, indicating that the socket has no more data to read.

`poll()` has the following prototype:

```
poll(struct pollfd fds[], nfds_t nfds, int timeout);
```

The `timeout` parameter can be set to `-1`, causing `poll()` to block indefinitely until at least one file descriptor becomes ready.

14. `epoll()`

`epoll` is an **event-driven I/O multiplexing** interface available on Linux.

Like `poll()`, it monitors file descriptors for events. However, instead of checking every file descriptor on each call, `epoll` returns only the file descriptors that are ready.

Instead of repeatedly looping over every monitored socket, we simply process the list of ready events returned by `epoll()`. This makes it much more scalable when handling a large number of connections.

To simplify the API, I wrapped the functions provided by `<sys/epoll.h>` inside an `epoll_handler` class.

15. Building the Server

The server follows these steps:

1. Resolve the server address.
 2. Create the listening socket.
 3. Bind the socket to the resolved address.
 4. Create an `epoll` instance.
 5. Create a `listener_connection` object containing the listener socket and the `epoll` file descriptor.
 6. Register the listener with `EPOLLIN | EPOLLET`.
 7. Enter the event loop and wait for ready file descriptors.
 8. If the ready file descriptor is the listener socket, accept the new client and register its socket with `epoll`.
 9. If a client disconnects, remove it from the `epoll` set and clean up its resources.
 10. If a client sends a message, broadcast it to all connected clients.
-

16. Building the Client

The client follows these steps:

1. Resolve the server address.
2. Create a socket.
3. Connect to the server.
4. Create a receiver thread that continuously calls `recv()`.
5. Use the main thread to read user input and send messages.
6. When the user enters `/quit`, call `shutdown()` on the client socket.

7. Wait for the receiver thread to exit by calling `join()`.
8. Finally, close the socket and terminate the program.